Probabilistic Forecasting with Anomaly Injection

CENG 463 – Introduction to Machine Learning – Progress Report

İzmir Institute of Technology, Spring 2026

Instructor: Prof. Dr. Aytuğ Onan

Group Members: Ozan Erdoğan, Student No. 290201102

1 Problem Definition

Task. I work with a multivariate weather time series $\{x_t\}_{t=1}^{T}$ where each $x_t \in \mathbb{R}^{14}$ is a vector of hourly meteorological variables. The model takes the past $L = 168$ hours (one week) of context and predicts the next $H = 24$ hourly air-temperature values. Two model families are compared. *Deterministic* models output a single point forecast in $\mathbb{R}^H$. *Probabilistic* models output a predictive distribution $p(y_{t+1:t+H} \mid x_{t-L+1:t})$ summarised by quantiles and prediction-interval coverage. After in-distribution evaluation, every model is re-evaluated under controlled *anomaly injection*: spikes, contextual outliers, level shifts, and FGSM-style perturbations are added to the test inputs at varying intensity. The question I want to answer is whether probabilistic models lose accuracy more slowly than deterministic ones, and whether their uncertainty actually rises around the injected anomalies.

Importance. Real weather forecasting systems often receive imperfect inputs due to sensor faults, missing updates, or unusual local conditions. Therefore, clean-data accuracy alone is not enough to judge whether a model is reliable in practice. This project evaluates how forecasting models behave under controlled input anomalies. This is important because useful uncertainty estimates can help users decide when a forecast should be trusted or treated with caution.

Dataset. The project uses the Jena Climate dataset, a multivariate weather time-series dataset containing meteorological variables such as air temperature, pressure, humidity, and wind-related measurements. The data is resampled to hourly frequency.

Inputs / outputs. Input: a 168-hour context window of 14 meteorological variables. **Output:** the next 24 hourly air-temperature values, either as a point forecast for deterministic models or as a predictive distribution for probabilistic models.

Task family. This project is a multistep time-series forecasting task with an additional robustness evaluation component. The main goal is to predict future temperature values from past weather observations. The anomaly-injection part extends the task by testing how stable the models remain when the input data becomes abnormal.

Challenges. The problem is technically challenging for several reasons. First, errors can accumulate over a 24-hour forecasting horizon, especially in autoregressive models. Second, probabilistic models must produce not only accurate forecasts but also well-calibrated uncertainty estimates, so their prediction intervals should contain the true values at the expected rate. Third, anomaly injection must be designed carefully: very small anomalies may have little effect, while unrealistic anomalies can make the task trivial. Therefore, the evaluation must consider both clean-data accuracy and robustness under abnormal inputs.

2 Dataset and Preprocessing Status

Dataset. Jena Climate 2009–2016, recorded at the Max Planck Institute for Biogeochemistry in Jena, Germany. The raw archive holds 420,551 observations spaced 10 minutes apart, with 14 variables covering pressure, temperature, dew point, humidity (relative, specific, and as water-vapour concentration), the saturation / actual / deficit family of vapour pressures, air density, wind speed, maximum gust, and wind direction.

Source and acquisition. The dataset is hosted on the TensorFlow CDN. `scripts/download_data.py` downloads the ∼13 MB zip, unpacks it, and writes the cleaned hourly version to `data/processed/jena_hourly.parquet`.

Preprocessing performed.

- Parsed the timestamps and resampled to hourly means: this turns the raw 420,551 rows into 70,128 hourly rows.
- The raw wind-speed columns occasionally store large negative sentinel values; these were turned into NaN before averaging.
- Around 2014-09-24 / 2014-09-25, the station logged no data for roughly three days. This gap is filled using time-aware linear interpolation, with forward/ backward filling at the edges, so the processed hourly series contains no missing values.
- The raw archive ends at 2016-12-31 23:50, which falls into the 2017-01-01 hour bucket; that incomplete bucket was dropped, and the series ends cleanly at 2016-12-31 23:00.
- Stored as Parquet so reload is fast.

After these preprocessing steps, the dataset is ready to be split and converted into forecasting windows for model experiments.

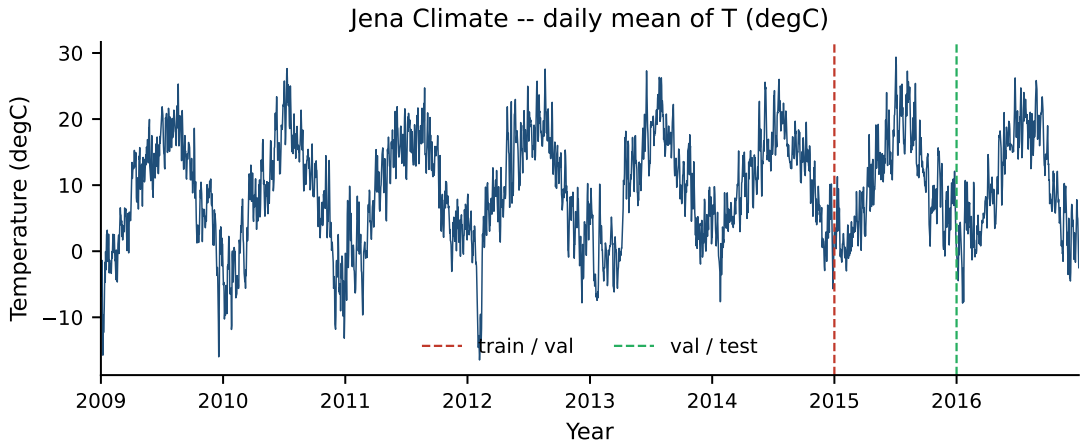


Figure 1: Daily-mean temperature across the full record, with the chronological train/validation/test boundaries (red: end of train, green: end of validation). Yearly seasonality is the dominant signal, and a mild upward trend is visible across years.

Splits (chronological, deterministic).

Split	Range	Hours
Train	2009-01-01 – 2014-12-31	∼52,584
Validation	2015-01-01 – 2015-12-31	∼8,760
Test	2016-01-01 – 2016-12-31	∼8,784

Target. The air-temperature channel `T (degC)`, with a range of roughly −25 to +38 °C across the record. The remaining 13 channels become exogenous covariates in later phases.

Standardisation and windowing. The target channel is standardised using statistics fitted on the training split only, and predictions are converted back to degrees before metric computation. For the LSTM baseline, sliding windows are constructed with lookback $L = 168$ and horizon $H = 24$. Training windows use stride 1, while validation and test windows use stride H to reduce overlap. Rolling-origin baselines such as ARIMA and seasonal naive forecasting are evaluated separately.

Issues encountered and resolved. An earlier hourly version still contained missing values from the September 2014 outage, which caused failures during metric computation. This was fixed by applying time-aware linear interpolation followed by forward/backward filling at the edges. The incomplete final hourly bucket was also removed so that the processed series ends cleanly at 2016-12-31 23:00. No class imbalance issue exists because the target is continuous air temperature rather than a discrete class label.

Preprocessing steps completed. After these preprocessing steps, the dataset has been downloaded, cleaned, resampled to hourly frequency, saved as Parquet, split chronologically, and prepared for baseline model experiments.

3 Literature Review Progress

Deterministic forecasting baselines. Hyndman and Athanasopoulos (*Forecasting: Principles and Practice*, 2021) present seasonal naive forecasting and ARIMA as standard reference methods for time-series forecasting. This supports the use of a seasonal naive model as a simple sanity-check baseline for hourly temperature, where daily seasonality is strong, and ARIMA as a classical statistical baseline. For the neural deterministic baseline, Hochreiter and Schmidhuber (1997) introduced the LSTM architecture, which is widely used for sequential data because it can model longer temporal dependencies than a simple recurrent network. In this project, the LSTM uses the same lookback and horizon setting as the later probabilistic models, giving a stronger deterministic reference point before evaluating DeepAR and the quantile-based Transformer.

Probabilistic forecasting. Salinas et al. (*DeepAR*, 2020) propose an autoregressive RNN that, at every step, outputs the parameters of a likelihood such as a Gaussian or Negative Binomial distribution and is trained by maximizing the likelihood. This motivates DeepAR as the parametric probabilistic baseline for this project. Wen et al. (*MQ-RNN*, 2017) take a different route: a multi-quantile forecasting head trained with the pinball loss, without making a fixed distributional assumption. This motivates the quantile-based Transformer baseline. Zhou et al. (*Informer*, 2021) and Wu et al. (*Autoformer*, 2021) are Transformer architectures specialized for long-horizon forecasting. Their experimental setup helps guide the choice of lookback, horizon, and channel handling so that the LSTM and Transformer models are evaluated under a consistent forecasting protocol.

Adversarial / anomaly evaluation. Goodfellow et al. (FGSM, 2015) and Madry et al. (PGD, 2018) provide the main background for gradient-based input perturbations. Although these methods were originally developed for adversarial attacks on classifiers, their perturbation-budget idea and ℓ_∞-norm constraint are useful for this project as a controlled way to define bounded input noise. In the planned anomaly-injection stage, an FGSM-style setting will be used as one type of input perturbation rather than as a full classifier attack.

For non-adversarial anomalies, Blázquez-García et al. (2021) summarize common time-series anomaly types such as point, contextual, and collective anomalies. This taxonomy fits the planned injection types in this project, including spikes, contextual outliers, and level shifts. Finally, Gneiting and Raftery (2007) motivates the use of proper scoring rules such as CRPS, which is important because the probabilistic models must be evaluated not only by point error but also by the quality of their predictive distributions.

What this means for my project. The reading suggests three concrete decisions. (1) Use seasonal naive, ARIMA, and LSTM as deterministic baselines before comparing against probabilistic models. (2) Use DeepAR as the parametric probabilistic model and a quantile-based Transformer as the non-parametric probabilistic model. (3) Evaluate the models not only with MAE and RMSE, but also with CRPS, pinball loss, prediction-interval coverage probability (PICP), and mean interval score (MIS). This allows the project to discuss both forecasting accuracy and uncertainty quality under clean and anomalous inputs.

4 Baseline Models

At this checkpoint, all three selected baseline models are implemented and can produce evaluation metrics. The numerical results are reported in the following section.

What I picked and why.

- **Naive Seasonal** with $S = 24$. This is the simplest baseline for an hourly series with a strong daily cycle: the forecast for each hour is the value from the same hour on the previous day. It serves as a sanity-check baseline that learned models should be able to beat.
- **ARIMA**$(2, 1, 2)$ with rolling-origin refitting. This is included as a classical linear statistical forecasting baseline. The order is fixed for now; order selection via auto-ARIMA is a Phase 2 item. I keep it non-seasonal in this phase so that the seasonal naive model separately captures the daily-cycle effect, while ARIMA provides a lightweight statistical reference point.
- **LSTM** ($L = 168$, $H = 24$, hidden 64, 2 layers, dropout 0.1, MSE loss, Adam 10^{-3}, 8 epochs). This is a small deterministic neural baseline. Its capacity is kept intentionally modest because the later anomaly experiments require a fast retraining loop, and the comparison should focus on deterministic versus probabilistic behaviour rather than simply on the effect of using a larger network.

Status. All three baselines are implemented under `src/baselines/` and are runnable from `scripts/run_*.py`. Each script writes a JSON file under `results/`. ARIMA refits every 50 origins on a moving 8000-sample window so that the rolling evaluation finishes in a reasonable time on a laptop.

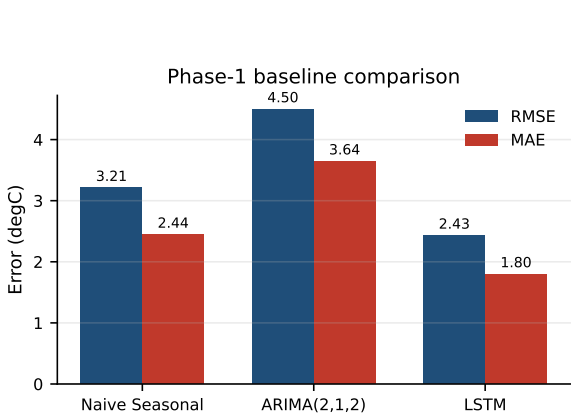
Implementation issues. Two implementation issues appeared during baseline evaluation. First, an earlier hourly Parquet file still contained missing values from the September 2014 outage, which caused the metric computation to fail. This was fixed in preprocessing by applying time-aware interpolation followed by forward/backward filling at the edges. Second, MAPE produced misleadingly large values, often above 100%, for every model. This happens because air temperature in Celsius can cross or approach zero in winter, making the relative-error denominator very small. Switching to Kelvin would only hide the issue: standardised or differenced models are invariant to a constant offset, so MAE and RMSE remain unchanged by the unit change, while MAPE changes artificially. Therefore, I keep MAE and RMSE as the primary metrics here and will add sMAPE and MASE in Phase 2.

5 Initial Experimental Results

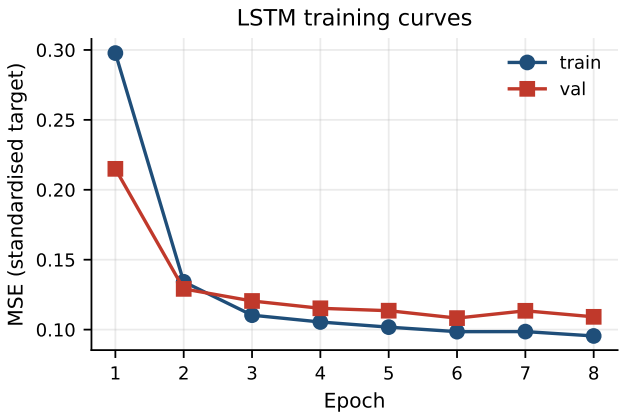
The numbers below are on the held-out 2016 test split, target `T (degC)`, horizon $H = 24$, predictions in original units. ARIMA is evaluated on the first 30 days of test using rolling origins because of refit cost; the other two baselines run over the full test split. The number of predictions differs across models because the LSTM is evaluated on non-overlapping 24-hour windows, while the rolling baselines generate forecasts over more origins.

Table 1: Phase-1 baseline forecasting results on Jena Climate. Lower is better.

Model	RMSE (°C)	MAE (°C)	#preds	Status
Naive Seasonal (S=24)	3.21	2.44	209,688	implemented
ARIMA$(2, 1, 2)$	4.50	3.64	16,728	implemented
LSTM (L=168, H=24, h= 64, 2 layers)	**2.43**	**1.80**	8,616	implemented



(a) Aggregate RMSE / MAE per baseline.



(b) LSTM training and validation loss (8 epochs).

Figure 2: Phase-1 baseline summary. The bar chart on the left makes the model ordering visible at a glance; the loss curves on the right suggest that the LSTM is not overfitting within eight-epoch.

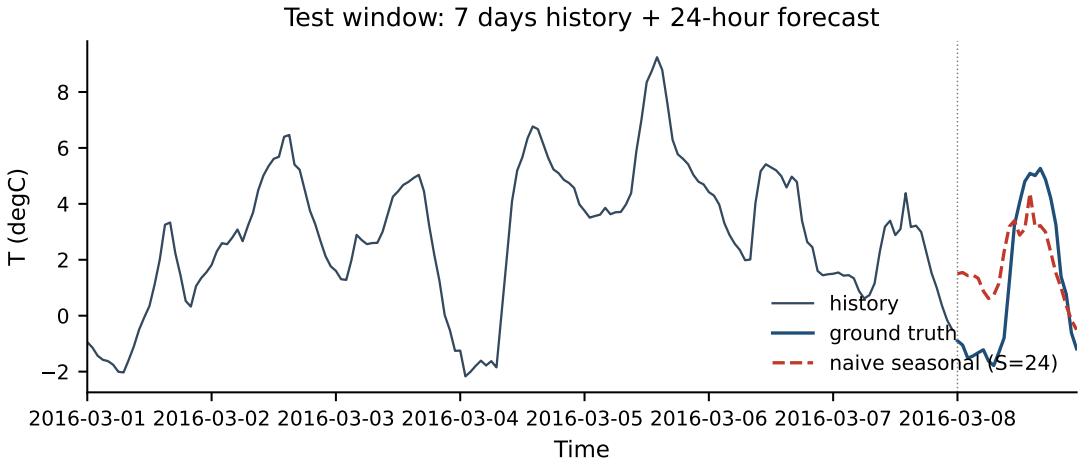


Figure 3: A representative test window from March 2016: seven days of history, then a 24-hour forecast. The seasonal-naive forecast (red dashed) shifts the previous day's hourly profile forward. It captures the diurnal phase almost exactly, but loses day-to-day amplitude variation, which is the part a learned model is expected to capture.

What the numbers say. Naive Seasonal lands at RMSE 3.21 °C and MAE 2.44. This is high but still reasonable for a one-line baseline on a series with a clear daily cycle. ARIMA$(2, 1, 2)$ performs worse, with RMSE 4.50 and MAE 3.64. The reason is structural: with $d = 1$ and no seasonal component, ARIMA captures short-range autocorrelation but misses the 24-hour cycle. SARIMA with a seasonal term should reduce this gap, and I will run it as a control in Phase 2. I keep the plain ARIMA result because it is a useful negative baseline: it shows that short-range autocorrelation alone is not enough for this series.

The LSTM beats both classical baselines, reaching RMSE 2.43 (−24% relative to seasonal naive) and MAE 1.80 (−26%). Training and validation losses fall from 0.30/0.22 at epoch 1 to 0.10/0.11 at epoch 8, with roughly the same shape on both curves and no late-stage divergence. The eight-epoch budget appears sufficient for this preliminary baseline.

What this means for Phase 2. The LSTM currently provides the strongest deterministic reference point at about 2.4 °C RMSE. A probabilistic model whose predictive mean is much worse than this reference would need to justify itself through substantially better uncertainty estimates. Therefore, DeepAR and the quantile Transformer should be evaluated not only by point forecasting accuracy, but also by whether their prediction intervals become more informative under anomalous inputs.

6 Planned Improvements and Technical Direction

The next phase extends the current baseline pipeline in three directions:

First, I will add two probabilistic forecasters: DeepAR, implemented as an autoregressive LSTM with a Gaussian or Student-t likelihood and NLL training, and a quantile-head Transformer trained with pinball loss over a fixed set of quantiles.

Second, I will extend the current target-only setup by using the remaining meteorological variables as exogenous covariates, so that the models can use the full multivariate weather context rather than only past temperature.

Third, I will evaluate every model under a common anomaly-injection protocol. The planned anomaly types are point spikes, contextual outliers, level shifts, and ℓ_∞-bounded FGSM-style perturbations on the input window. Probabilistic-specific metrics such as CRPS, pinball loss, prediction-interval coverage probability (PICP), and mean interval score (MIS) will be added on top of MAE and RMSE. For relative error reporting, sMAPE and MASE will replace MAPE. I also plan to add SARIMA$(p, d, q)(P, D, Q)_{24}$ as a control model to check whether the poor plain-ARIMA result is mainly due to the missing seasonal component rather than a coding issue.

7 Ablation and Error Analysis Plan

Ablation study. The ablation study will compare several parts of the pipeline to understand their contribution. For the input setup, I will compare target-only forecasting against multivariate forecasting with covariates. For temporal context, I will test different lookback lengths, such as $L \in \{72, 168, 336\}$. For the deep models, I will vary hidden capacity and regularization. For DeepAR, I will compare Gaussian versus Student-t likelihoods. For the quantile Transformer, I will compare smaller and larger quantile sets, such as 3 versus 7 quantiles. Finally, I will perform per-anomaly-type ablations to see which injection mode each model is most sensitive to.

Error analysis. The error analysis will examine where and why the models fail. I will break down MAE/RMSE by forecast horizon to see whether errors grow smoothly or increase sharply at later hours. I will also compare errors across seasonal contexts, such as month or temperature range, because winter near-zero temperatures already caused issues for percentage-based metrics. Under anomaly injection, the worst failure windows will be grouped by anomaly type and intensity. For probabilistic models, I will specifically examine overconfident failures, where the prediction interval is narrow but misses the true value, because this is the most dangerous failure mode for downstream use.

8 Visualization and Interpretation Plan

A first pass is already included in this report: Figure 1 shows the time series with chronological split markers, Figure 2 compares baseline errors and LSTM training curves, and Figure 3 shows a representative 24-hour forecast window.

In the final phase, I plan to add visualizations that directly support the main research question. Forecast plots with prediction intervals will show whether probabilistic models widen their uncertainty around anomalous inputs. Calibration plots, reliability curves, and PIT histograms will show whether the predicted uncertainty is statistically meaningful. Per-horizon RMSE and CRPS curves will show how error changes across the 24-hour forecast horizon. A model-versus-anomaly heatmap will summarize robustness degradation across anomaly types and intensities. For the Transformer model, attention maps may be used as a sanity check to see whether the model attends to seasonally-aligned positions, although these will be interpreted cautiously rather than treated as a complete explanation.

9 GitHub and Reproducibility Status

Repository. The project repository is available at https://github.com/ozanerdogan/prob-forecast-anomaly.

Current structure. The `src/` directory contains the main data and modelling code, including `data_loader.py`, `preprocessing.py`, `metrics.py`, and the `src/baselines/` package. The baseline package currently includes `naive_seasonal.py`, `arima_baseline.py`, and `lstm_baseline.py`. The `scripts/` directory contains the runnable entry points: `download_data.py`, `run_naive.py`, `run_arima.py`, `run_lstm.py`, and `make_figures.py`. The `data/` directory contains dataset acquisition instructions, while raw and processed data are gitignored. The `results/` directory stores JSON metric files and generated figures, and `report/` stores the LaTeX source and compiled report.

Dependencies and README status. The repository includes a `requirements.txt` file for the current Python environment. It lists the major dependencies used so far, including `numpy`, `pandas`, `scikit-learn`, `statsmodels`, `torch`, `matplotlib`, `seaborn`, `tqdm`, `pyyaml` and `pyarrow`. The `README.md` documents the current workflow: create a virtual environment, install dependencies, download and preprocess the dataset, run the baseline scripts, and regenerate the figures.

Running the current experiments. The current pipeline can be reproduced with the following commands:

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

python scripts/download_data.py
python scripts/run_naive.py
python scripts/run_arima.py
python scripts/run_lstm.py
python scripts/make_figures.py
```

The dataset preparation step is fully scripted. The raw Jena Climate archive is downloaded, cleaned, resampled to hourly frequency, and saved as `data/processed/jena_hourly.parquet`. Raw and processed data are not committed to GitHub.

Reproducibility details. Each baseline script writes a JSON file under `results/` that contains the evaluation metrics and relevant model configuration. The LSTM training script uses a fixed random seed with `torch.manual_seed(42)`. This makes the current baseline runs reasonably reproducible, although small differences may still occur depending on hardware and PyTorch backend behavior.

Commit history. The repository is currently at an early stage, with the initial baseline pipeline, preprocessing code, figures, and report already uploaded. The remaining project components will be added through incremental commits during the final development phase.

10 Current Challenges and Next Steps

Current challenges. At this stage, computation is manageable, but it may become a constraint in the next phase. The current LSTM baseline can be trained in a reasonable time, yet the planned probabilistic models, anomaly-intensity sweeps, and ablation experiments will require many more training and evaluation runs. For this reason, I may need to use smaller debugging configurations, limit the number of repeated runs, or use GPU access for the full final evaluation.

A second challenge is anomaly realism. If anomaly intensity is fixed in absolute temperature units, the same perturbation may have different meaning in winter and summer. To make the injected anomalies more comparable across the year, I plan to scale anomaly intensity using a local rolling standard deviation. This should make the robustness tests less dependent on the seasonal temperature level.

A third challenge is probabilistic calibration. Even if a probabilistic model gives good point forecasts, its nominal prediction intervals may not achieve the expected empirical coverage. For this reason,

validation-set calibration will be considered in Phase 2, possibly using a simple post-hoc adjustment such as isotonic calibration or temperature scaling.

Next steps. Before the final submission, I plan to complete the probabilistic part of the pipeline by implementing DeepAR and a quantile-based Transformer. I will also add SARIMA as a seasonal statistical control, since the current plain ARIMA baseline performs poorly mainly because it does not model the 24-hour cycle. After that, I will add the anomaly-injection harness with spikes, contextual outliers, level shifts, and FGSM-style bounded input perturbations.

The final evaluation will compare deterministic and probabilistic models on both clean and anomalous test inputs. In addition to MAE and RMSE, I will report probabilistic metrics such as CRPS, pinball loss, prediction interval coverage probability (PICP), and mean interval score (MIS). I will also complete the planned ablation study, error analysis, and visualizations, then update the final report with the full experimental results.